

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Computer Vision with OpenCV **COURSE CODE:**

DSA0202 COURSE OUTCOME COVERED

CO3: Analyze shape representation methods and techniques such as contours, regions, deformable models, and multi-resolution analysis. (BL4)

Assessment Tool Weightage: 50%

QUESTIONS: 5 Total Marks:50 Annexure A

EXPERIMENT

Code of Conduct:

I M Tilak Venkata Sai(192424415)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: tilak sai

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts	
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	

Docu- mentat ion	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation	
---------------------	----	---	--	---	--	--

Annexure A

EXPERIMENT

Experiment 1: Contour-Based Representation & Fourier Descriptors

Scenario: Represent 2D shapes using contours and Fourier descriptors.

Input Data:

- A. Binary images of 2D hand-drawn letters (A–Z) or simple symbols (triangle, square, star)
- B. Image size: 256 × 256 pixels
- C. File format: PNG or BMP

Experiment 2: Region-Based Representation & Medial Axis

Scenario: Analyze object shapes in binary images using medial axis. **Input Data:**

- A. Binary images of industrial components (e.g., gears, machine parts)
- B. Image size: 512 × 512 pixels
- C. Defects like small holes, protrusions, or missing parts included
- D. File format: PNG

Experiment 3: Deformable Curves and Snakes (Active

Contours) Scenario: Segment organs or objects with irregular boundaries. **Input Data:**

- A. Grayscale medical images (e.g., liver or heart slices)
- B. Image size: 512 × 512 pixels
- C. Noise: mild Gaussian noise
- D. File format: DICOM or PNG

Experiment 4: Level Set Methods

Scenario: Track moving objects with evolving boundaries.

Input Data:

- A. Video sequence (30 frames) of a moving object (e.g., a bouncing ball or

moving hand)

B. Resolution: 640×480 pixels

C. Format: MP4 or sequence of PNG images

Experiment 5: Multi-Resolution & Wavelet Descriptors

Scenario: Shape recognition at multiple scales.

Input Data:

A. Grayscale images of objects at different scales (e.g., leaves, mechanical parts, coins)

B. Image sizes: 128×128 , 256×256 , 512×512 pixels

C. Format: PNG

Experiment 1: Contour-Based Representation & Fourier Descriptors

Code:

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

img = cv2.imread("/content/download (4).jpg", 0)

_, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)

contours, _ = cv2.findContours(
    binary, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE
)

contour = max(contours, key=cv2.contourArea)

# Convert contour to complex coordinates
points = contour[:, 0, :]
complex_points = points[:, 0] + 1j * points[:, 1]

# Fourier Transform
fourier = np.fft.fft(complex_points)

# Keep only important descriptors
keep = 30

fd = np.zeros_like(fourier)

center = len(fourier) // 2

fd[:keep] = fourier[:keep]

fd[-keep:] = fourier[-keep:]
```

```

# Reconstruct contour
reconstructed = np.fft.ifft(fd)

# Display
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)

plt.imshow(binary, cmap="gray")

plt.title("Original Binary Image")

plt.axis("off")

plt.subplot(1, 2, 2)

plt.plot(reconstructed.real, reconstructed.imag)

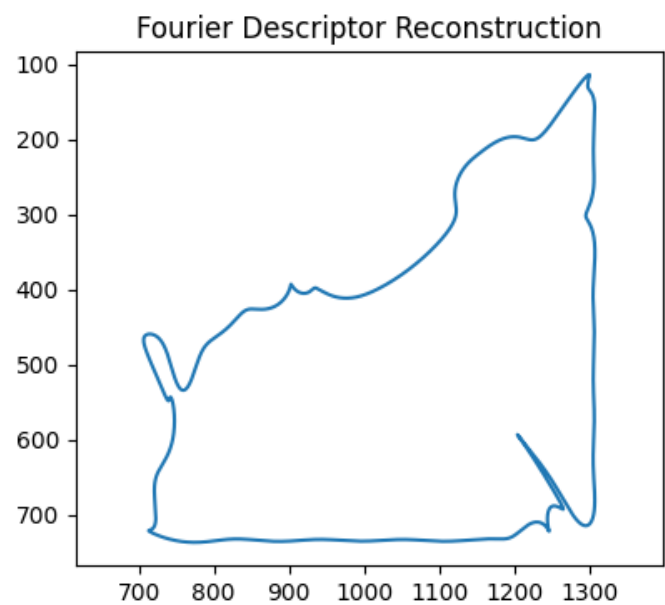
plt.gca().invert_yaxis()

plt.title("Fourier Descriptor Reconstruction")

plt.axis("equal")

plt.show()

```



Experiment 2: Region-Based Representation & Medial Axis

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

from skimage.morphology import medial_axis

img = cv2.imread("/content/download (4).jpg", 0)

_, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)

# Convert to Boolean
binary_bool = binary > 0

# Medial axis
skeleton, distance = medial_axis(binary_bool, return_distance=True)

# Create output
result = np.zeros_like(binary)
result[skeleton] = 255

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(binary, cmap="gray")
plt.title("Original Component")
```

```
plt.axis("off")
```

```
plt.subplot(1, 2, 2)
```

```
plt.imshow(result, cmap="gray")
```

```
plt.title("Medial Axis / Skeleton")
```

```
plt.axis("off")
```

```
plt.show()
```

Original Component



Medial Axis / Skeleton



Experiment 3: Deformable Curves / Active Contours

```
import cv2
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from skimage.segmentation import active_contour
```

```
img = cv2.imread("/content/download (4).jpg", 0)
```

```
# Resize if required
```

```
img = cv2.resize(img, (512, 512))
```

```
# Gaussian noise reduction  
img = cv2.GaussianBlur(img, (5, 5), 0)
```

```
# Initial circular contour  
center = (256, 256)  
radius = 150
```

```
theta = np.linspace(0, 2 * np.pi, 300)
```

```
x = center[0] + radius * np.cos(theta)  
y = center[1] + radius * np.sin(theta)
```

```
init = np.array([y, x]).T
```

```
# Active contour  
snake = active_contour(  
    img,  
    init,  
    alpha=0.01,  
    beta=10,  
    gamma=0.001,  
    max_num_iter=2500  
)
```

```
plt.figure(figsize=(8, 8))
```



```
plt.imshow(img, cmap="gray")
```

```
plt.plot(init[:, 1], init[:, 0], "--r", label="Initial Contour")
```

```
plt.plot(  
    snake[:, 1],  
    snake[:, 0],  
    "-b",  
    linewidth=2,  
    label="Active Contour"  
)
```

```
plt.title("Active Contour / Snake")
```

```
plt.legend()
```

```
plt.axis("off")
```

```
plt.show()
```



Experiment 4: Level Set Methods

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

from google.colab.patches import cv2_imshow # Import for displaying images in Colab


cap = cv2.VideoCapture("/content/lv_0_20260609232328.mp4")


# Check if the video file was opened successfully
if not cap.isOpened():
    print("Error: Could not open video file. It might be corrupted or in an unsupported format.")
else:
    ret, frame = cap.read()

    if not ret:
        print("Video not found or unable to read the first frame.")
    else:
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        # Initial threshold
        _, mask = cv2.threshold(gray, 100, 255, cv2.THRESH_BINARY)

        # Find object
        contours, _ = cv2.findContours(
            mask,
```

```
cv2.RETR_EXTERNAL,  
cv2.CHAIN_APPROX_SIMPLE  
)
```

```
if len(contours) == 0:
```

```
    print("Object not detected")
```

```
else:
```

```
    contour = max(contours, key=cv2.contourArea)
```

```
# Example: process only a few frames or iterate completely
```

```
# You can add a counter here to limit the number of frames processed
```

```
# frame_count = 0
```

```
while True:
```

```
    ret, frame = cap.read()
```

```
    if not ret:
```

```
        break
```

```
    # frame_count += 1
```

```
    # if frame_count > 100: # Process 100 frames then stop
```

```
    #     break
```

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

```
_, mask = cv2.threshold(  
    gray,
```

```
100,  
255,  
cv2.THRESH_BINARY  
)  
  
# Morphological processing  
kernel = np.ones((5, 5), np.uint8)  
  
mask = cv2.morphologyEx(  
    mask,  
    cv2.MORPH_CLOSE,  
    kernel  
)  
  
contours, _ = cv2.findContours(  
    mask,  
    cv2.RETR_EXTERNAL,  
    cv2.CHAIN_APPROX_SIMPLE  
)  
  
if len(contours) > 0:  
  
    contour = max(  
        contours,  
        key=cv2.contourArea
```

```
)
```

```
cv2.drawContours(
```

```
    frame,
```

```
    [contour],
```

```
    -1,
```

```
    (0, 255, 0),
```

```
    2
```

```
)
```

```
# Use cv2_imshow for displaying frames in Colab
```

```
cv2_imshow(
```

```
    frame
```

```
)
```



Experiment 5: Multi-Resolution & Wavelet Descriptors

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

import pywt

img = cv2.imread("/content/download (4).jpg", 0)

# Create different resolutions

img128 = cv2.resize(img, (128, 128))

img256 = cv2.resize(img, (256, 256))

img512 = cv2.resize(img, (512, 512))

images = [img128, img256, img512]

titles = ["128 x 128", "256 x 256", "512 x 512"]

plt.figure(figsize=(12, 4))

for i in range(3):

    plt.subplot(1, 3, i + 1)

    plt.imshow(images[i], cmap="gray")

    plt.title(titles[i])

    plt.axis("off")

plt.show()
```

```
# Wavelet decomposition

coefficients = pywt.dwt2(img256, "haar")

LL, (LH, HL, HH) = coefficients

plt.figure(figsize=(10, 8))

plt.subplot(2, 2, 1)
plt.imshow(LL, cmap="gray")
plt.title("LL - Approximation")

plt.subplot(2, 2, 2)
plt.imshow(LH, cmap="gray")
plt.title("LH - Horizontal Details")

plt.subplot(2, 2, 3)
plt.imshow(HL, cmap="gray")
plt.title("HL - Vertical Details")

plt.subplot(2, 2, 4)
plt.imshow(HH, cmap="gray")
plt.title("HH - Diagonal Details")

plt.tight_layout()
```

plt.show()

